

Pygame Notes

Setting up

`"import pygame"`

`"from pygame.locals import *"` → removes need to do `"pygame.()"` for constants

`"pygame.init()"` → initialises pygame

`"pygame.font.init()"` → initializes fonts

Setting up window

`"screen = pygame.display.set_mode((x, y))"` → game window

`"screen = pygame.display.set_mode((x, y), pygame.RESIZABLE)"` → resizable

`surface.fill(color)` → sets surface colour

`"x, y = screen.get_size()"` → gets size of screen

`"pygame.display.set_caption("Caption Name")"` → Sets window name

`Icon = pygame.image.load('logo.png')` → loads image

`pygame.display.set_icon(Icon)` → sets icon to image

Game Loop:

`running = True`

`while running:` → must be while loop

`for event in pygame.event.get():` → iterates through every event that occurred

`if event.type == pygame.QUIT:` → if the program quit, stop the program

`running = False`

`pygame.quit()` → Quits game at end of loop

Surfaces

Sprite → A 2D object (like our square) displayed on the screen.

Surface → A canvas for drawing (even the screen is a Surface)

Rect → A rectangle object for positioning and collisions.

`"pygame.surface(tuple of width&height, flags, depth, mask)"`

 Flags → optional Special options that change how the surface behaves.

 Depth → optional how many bits per pixel

 Mask → optional, how colour bits are stored (Advanced)

```
"pygame.surface_name.fill(color, rect=None, special_flags=0)"
```

Rect → place to include a rectangle to only fill that area (needs shape (x, y, width, height) or rect object)

```
"pygame.surface_name.blit(source, dest, area=None, special_flags=0)"
```

Source → what to blit

Dest → coords

Area → lets you cut out a portion of the source and only show that part

Special Flags → controls blending effects

```
"pygame.surface_name.set_colorkey(Color, flags=0)" → only the colour in the key will show if blitted
```

```
"pygame.surface_name.get_colorkey()" → gets the colour key of a surface if there are one
```

```
"pygame.surface_name.set_alpha()" → sets alpha (255 is opaque, 0 is transparent)
```

```
"pygame.surface_name.get_alpha()" → gets alpha of surface
```

```
"pygame.surface_name.convert(Surface=None)" → makes a copy of surface with changed format
```

```
"pygame.surface_name.convert_alpha(Surface)" → creates a new copy of the surface with the desired pixel format
```

```
"pygame.surface_name.copy()" → make a copy with same pixel formats
```

Classes/Sprites

```
class Sq(pygame.sprite.Sprite): → class - a blueprint for an asset that all assets have
```

```
    def __init__(self):
```

```
        super().__init__() → runs the "Sprite" code that was imported for blueprint
```

```
        self.surf = pygame.Surface((25, 25)) → adds surface/size for class
```

objects

```
        self.surf.fill((0, 200, 255)) → adds colour
```

```
"pygame.sprite.Sprite" → Calling for asset "Sprite" within the "sprite" folder of the Pygame library
```

```
"self" → placeholder variable referring to specific object when called later
```

```
"__init__" → sets up an object when created
```

```
"s1 = Sq()" → assigns Sq() class to s1, adding properties
```

Within the game loop code:

```
    "screen.blit(s1.surf, (40, 40))" → draws
```

```
".blit()" → draws surface (assigned from "self.surf") onto screen surface
```

```
"pygame.display.flip()" → refreshes screen to update drawings
```

```
"pygame.display.update(object)" → refreshes specific object
```

Shapes/Objects

Rectangles

Rectangles have attributes:

- rect.x → x position
- rect.y → y position
- rect.left → left edge position
- rect.right → right edge
- rect.top → top edge
- rect.bottom → bottom edge

`pygame.draw.rect(which surface, color, an object or position & size, optional border width)` → how to draw a rectangle, if border width > 0 → rect will be empty with only border outline

`"if rect_1.colliderect(rect_2):"` → checks if 2 rectangles are colliding

`"surface_name.get_rect()"` → makes a surface a rectangle with rectangle properties

`pygame.draw.rect(surface, color, rect, width=0, border_radius=0, border_top_left_radius=-1, border_top_right_radius=-1, border_bottom_left_radius=-1, border_bottom_right_radius=-1)` → for rounder corners

`rect.topleft = (20, 20)` → sets top left of rectangle to location

`"rect.move_ip(-8,0)"` → moves object.

`inflate_ip()` → positive, grow. Negative, shrink

`clamp_ip(<other rect>)` → forces a rect inside another

`union_ip()` → groups

`unionall_ip(list of rectangles)` → groups all rectangles

`fit_ip()` → resizes rect to fit into another

`<method>()` → gives you a NEW rect

`<method>_ip()` → edits the SAME rect

Circles/ellipse

`"pygame.draw.circle(which surface, color, centre position, radius, optional border width)"` → draws shapes, same border rules as rectangle

`"pygame.draw.ellipse(surface, color, position&size, width=0)"` → draws an ellipse

Polygons

`"polygon(surface, color, each coordinate of points, width)"` → draws any polygon, same border rules as other shapes

Lines

`pygame.draw.line(surface, color, start_pos, end_pos, width)` → draws lines, width is not optional for line

Text

`pygame.font.get_init()` → checks if font has been initialized

`font1 = pygame.font.SysFont('freesanbold.ttf', 50)` → Get a font file and font size

`_text = font1.render("Text", True, (255, 255, 255))` → renders text, "True" means vector not pixels

`screen.blit(_text, (10, 10))` → used within game loop, adds text on surface with position

`textRect1 = _text1.get_rect()` → giving you a rectangle that matches the size of your surface (image/text) so you can easily position it.

`textRect1.center = (250, 250)` → Sets centre of text

`display_surface.blit(_text1, textRect1)` → Blits rectangle text

Importing images

`image = pygame.image.load("Screenshot.png")` → imports image

`BASE_DIR = os.path.dirname(__file__)` → gets file directory

`canvas.blit(image, dest = position)` → within game loop, draws image and uses "dest" for position

`pygame.transform.scale(img, (100, 100))` → changes image size

Importing audio

`mixer.init()` → imports mixer

`mixer.music.load("song.mp3")` → Import song

`mixer.music.set_volume(0.7)` → set volume

`mixer.music.play()` → Play song

`mixer.music.pause()` → pause

`mixer.music.unpause()` → unpause

`mixer.music.stop()` → stop music

Event Handling - Game loop

`pygame.time.wait(2000)` → pauses (in milliseconds)

`pygame.time.delay` → more accurate pause by using processor

`Pygame.time.get_ticks()` → gives time which has been running

When 1 value (lets say x) is % to another value (let's say y) then when increasing x by 1, the result will be 1, 2, 3, 4, 5... till it reaches y. Then when it reaches y, it will return 0 then increase one by one till it reaches y again.

Events

SR No.	Event	Attributes
1.	KEYDOWN	key, mod, unicode
2.	KEYUP	key, mod
3.	MOUSEBUTTONUP	pos, button
4.	MOUSEBUTTONDOWN	pos, button
5.	MOUSEMOTION	pos, rel, buttons
6.	QUIT	-

```
for e in pygame.event.get():  
    if e.type == pygame.KEYDOWN: → one event per run, not meant for holding (e.g. jump,  
    pause, etc)  
        if e.key == pygame.K_SPACE: → once type of event specified, key can be called
```

pygamekey	Description
K_BACKSPACE	backspace
K_TAB	tab
K_CLEAR	clear
K_RETURN	return
K_PAUSE	pause
K_ESCAPE	escape
K_SPACE	space
K_EXCLAIM	exclaim
K_HASH	hash
K_QUOTEDBL	quotedbl
K_DOLLAR	dollar
K_AMPERSAND	ampersand
K_QUOTE	quote
K_LEFTPAREN	left parenthesis
K_RIGHTPAREN	right parenthesis
K_ASTERISK	asterisk
K_PLUS	plus sign
K_COMMA	comma
K_MINUS	minus sign
K_PERIOD	period
K_SLASH	forward slash
K_0	0
K_1	1
K_2	2
K_3	3
K_4	4
K_5	5
K_6	6
K_7	7
K_8	8
K_9	9
K_COLON	colon
K_SEMICOLON	semicolon
K_LESS	less-than sign
K_EQUALS	equals sign
K_GREATER	greater-than sign
K_QUESTION	question mark
K_AT	at
K_LEFTBRACKET	left bracket
K_BACKSLASH	backslash
K_RIGHTBRACKET	right bracket
K_CARET	caret
K_UNDERSCORE	underscore
K_BACKQUOTE	grave
K_a,b,c...z	A to Z Alphabet

K_KP_EQUALS	keypad equals
K_UP	up arrow
K_DOWN	down arrow
K_RIGHT	right arrow
K_LEFT	Left arrow
K_INSERT	Insert
K_HOME	Home
K_END	End
K_PAGEUP	Page Up
K_PAGEDOWN	Page Down
K_F1, K_F2, K_F3.....K_F15	F1 to F15
K_NUMLOCK	Numlock
K_CAPSLOCK	Capslock
K_SCROLLLOCK	Scrollock
K_RSHIFT	Right shift
K_LSHIFT	Left shift
K_RCTRL	right control
K_LCTRL	Left control
K_RALT	Right alt
K_LALT	Left alt
K_RMETA	right meta
K_LMETA	left meta
K_LSUPER	left Windows key
K_RSUPER	right Windows key
K_MODE	mode shift
K_HELP	Help
K_PRINT	Print Screen
K_SYSREQ	sysrq
K_BREAK	Break
K_MENU	Menu
K_POWER	Power
K_EURO	Euro

“event.pos” → gets position of event

For letter: `pygame.K_{lowercase letter}`
For command: `pygame.KMOD_{command eg. RSHIFT}`

```
elif event.type == pygame.MOUSEMOTION: → when user moves mouse
    if event.rel[0] > 0: → rel[0] means x value, positive right, negative left
    if event.rel[1] > 0: → rel[1] means y value, positive down, negative up
```

`print(event.rel)` → prints position of mouse

```
elif event.type == pygame.MOUSEBUTTONDOWN: → when user clicks
    if event.button == 3: → checks which type of input (see table below)
```

SR No.	Button	Value
1.	Left mouse button	1
2.	Mouse wheel button	2
3.	Right mouse button	3
4.	Mouse wheel scroll up	4
5.	Mouse wheel scroll down	5

```
elif event.type == pygame.MOUSEBUTTONUP: → Mouse released
```

Custom Events

`<event_name> = pygame.USEREVENT + 1` → Makes an event with a new ID

`event = pygame.event.Event(<event_name>, additional info)` → makes event into event type

Additional info → change in variable, etc

`pygame.event.post(event)` → trigger event with if statement condition

`pygame.event.post(pygame.event.Event(ON_BOX))` → combine both lines

`if box.collidepoint(pygame.mouse.get_pos()):` → if an object collides with mouse position

`pygame.time.set_timer(event, duration)` → set an event with time interval

Detect

```
for event in pygame.event.get():
    if event.type == ADD_EVENT:
        print("Custom event triggered!")
```

Keys pressed (continuous)

`"keys = pygame.key.get_pressed()"` → opens functions, used for continuous movement/holding
`if keys[pygame.K_LEFT]:` → indexes pygame function to check key (not in an event if statement)

`mouse = pygame.mouse.get_pressed()` → for mouse click and hold
`if mouse[0]:` → left mouse button

`event.unicode` → converts key pressed unicode to actual character

Clock

`clock = pygame.time.Clock()` → initialises clock

`clock.tick(60)` → used within game loop, limits to 60fps

`get_time()` → number of milliseconds between 2 ticks

`get_fps()` → returns fps of clock